

Ler e escrever são uma operação; ordenar não é essa --- ela precisa de uma referência , e a referência são os naturais , construídos em oito degraus que são todos involuções. Uma sequência é uma régua , e ordená-la é levá-la ao seu ponto fixo --- a forma canónica --- de onde se converte para qualquer outra. É esse mapa entre régua que aqui se constrói e se valida, com as peças da teoria da dualidade: a estaca $x \rightarrow x^*$, a Lei 2 $-f = f^{-1}$, a bidualidade $J^4 = \text{id}$ e o trial . Nenhum elemento é comparado com outro. A permutação é o dual , e guardá-la torna o processo invertível : a volta é exata e o apagamento é zero . O critério não é a lista sair crescente --- é a volta fechar.

A referência são os naturais

Ler e escrever são uma operação --- o MOVE, com o sinal a decidir o sentido. Ordenar não é essa: precisa de uma referência , e a referência são os naturais . O catálogo constrói-a em oito degraus, todos medidos e todos com o mesmo mecanismo --- uma involução com $v \circ v = \text{id}$ e ponto fixo na fronteira: $\{ \mapsto n \mapsto z \mapsto 1/z \mapsto +^* \}$

E cada degrau paga: $\rightarrow$ ganha o dual da soma, $\rightarrow$ o dual do produto, e $\rightarrow$ ganha a raiz de -1 e perde a ordem . Em todo o quadrado é não negativo, e é isso que sustenta a ordem; com i há um quadrado negativo e ela não pode existir. A ordem vive de até e não sobe mais --- e é por isso que a referência são os naturais, e não um corpo qualquer.

O natural nasce na transição, e cada involução deixa uma marca

O natural não está no valor . Ele aparece quando se passa de uma dimensão para a seguinte, e cada involução do relógio deixa uma marca :

@ccc@

transição & involução & marcas

6-5 & J & 1

5-4 & J & 2

4-3 & J & 3

3-2 & J & 4

2-1 & J & 5

Cinco transições, cinco marcas --- uma por dimensão atravessada --- e o período do relógio fecha em quatro em todas elas: a escada muda a fase, não o fecho.

E o relógio lê, em vez de perguntar

Uma colisão é uma marca do contador. Cada elemento marca o seu natural ao entrar, e ler é seguir as marcas --- os naturais que ninguém marcou não são visitados. Numa sequência de 16 elementos sobre 4096 naturais possíveis, o percurso visita 14 de 4096 e faz 0 perguntas.

E a única pergunta feita é de igualdade , nunca ``qual é maior".

a bijeção valor natural é injetiva sem colisões; o percurso visita 14 de 4096 e faz 0 perguntas de ordem; $v \circ v = \text{id}$ nos 8193 naturais duais com um ponto fixo; a decrescente é a crescente ao espelho, elemento a elemento; e o controlo : em nenhum quadrado é negativo, e com i há um --- a ordem não sobe (ests/ordena_n.c).

As peças

Todas as peças estão demonstradas na Teoria e nenhuma é importada.

[a estaca] Num conjunto de centro c , a estaca é $[x^\wedge = 2c - x]$. Ela é involutiva : $x^\wedge = x$. A sua fronteira é o conjunto dos seus pontos fixos, $x^\wedge = x$, isto é, $x=c$ --- e ele é um só.

A estaca troca de lado. O que ela fixa é o centro, e é por isso que ele não pertence a nenhum dos lados: é a fronteira entre eles .

[a Lei 2 e o J] A Lei 2 escreve-se, onde o dual é a inversa, $[-f = f^\wedge^{-1}]$, o que força $f^2 = -1$. Em duas coordenadas isso é $J(a,b)=(-b,a)$: uma rotação de um quarto.

[a bidualidade fecha] $J^2 = -1$ e $J^4 = id$. A órbita de J visita quatro quadrantes distintos e volta ao primeiro.

Aplicando J duas vezes a (a,b) obtém-se $(-a,-b)$, que é -1 vezes o par; aplicando quatro, (a,b) . Como J leva cada quadrante ao seguinte e J^4 é a identidade, os quatro são distintos e o quinto passo é o primeiro.

É este fecho que garante a terminação, e não uma condição imposta por fora.

O algoritmo

Seja A uma lista finita. Se $|A| \leq 1$, ela já está ordenada. Caso contrário, tome-se $c = A + (A - A)/2$ e classifique-se cada elemento pelo sinal da estaca : $[x^\wedge > x \ x < c, \ x^\wedge < x \ x > c, \ x^\wedge = x \ x = c]$

São três, e não dois --- é o trial . A esquerda e a direita descem outra vez; a fronteira fica entre elas, e não desce, porque não há para onde: ela é o ponto fixo.

@||@

o que a peça faz & no algoritmo

a estaca é involutiva & garante que os dois lados são imagem um do outro

a fronteira é o ponto fixo & é ela que parte, e não pertence a nenhum lado

o trial & três destinos: esquerda, fronteira, direita

a bidualidade & o fecho da órbita, e portanto a terminação

Não se compara nada

O destino de um elemento sai do sinal de $x^\wedge - x$ --- uma leitura sobre o próprio elemento --- e nunca de uma pergunta sobre outro elemento. Em por-unidade não há grandeza a comparar: há posição a ler, e ela já está no elemento.

O que se mede

a estaca é involução em 101 pontos e o seu ponto fixo é um só; ela parte em 50 e 50, e o dual de cada elemento cai sempre no outro lado; o J visita os quatro quadrantes, todos distintos, e volta ao quarto; $J^2 = -1$ e $J^4 = id$ em 169 pares; 200 listas saem crescentes e completas descendo pela estaca (tests/dualsort.c).

E o que acontece quando se tira uma peça

Três mutações, e nenhuma delas degrada o resultado quebra-no .

@||@

tirar & o que acontece

o ponto fixo vai para um dos lados & a partição deixa de reduzir --- não termina
o sinal do J ($J^2 \neq -1$) & duas asserções falham
~~a estaca sem ponto fixo & tudo cai do mesmo lado --- não termina~~

Duas das três não dão resultado errado: não dão resultado nenhum. É o teste mais forte
de que a lei não é decoração --sem ela não há algoritmo .

A cruz, e por que quatro bastam

Ordenar pares muda o que se pode dizer, e a razão é a torre: em R^n a ordem já está na
estrutura. Num número solto há uma coordenada e a órbita das involuções degenera ; num par
há duas, e é aí que a bidualidade se vê.

A estaca move; a cruz mede o que não se moveu , e tem duas coordenadas:
 $x \oplus x^{\wedge} = 2c$ && (invariante ---a que mede),
 $x \otimes x^{\wedge}$ && (varia, máxima no ponto fixo ---a que ordena).

A soma não distingue nada: vale $2c$ em todo o conjunto. O produto satura exatamente em $x=c$,
e o ponto fixo é essa saturação --- a passagem da dimensão. A cruz lança a projeção no
dual , e é ela que dá a segunda involução.

Com pares, as duas são independentes e a órbita dá quatro saídas distintas. Quatro
bastam; não foi preciso derivar oito.

a soma $x \oplus x^{\wedge}$ é invariante em 11 pontos e vale $2c$; o produto é máximo em $x=c$; e ordenar
pares com uma estaca por coordenada dá 4 saídas distintas de 4 --- com a mutação (as duas
estacas iguais) a cair para 2, que é a degenerescência (tests/dualsort.c).

Comparado com os outros

E a régua não é o relógio. O sistema é reversível: não tem tempo, só latência --- e latência é
da máquina, não do algoritmo. A régua é Landauer : bits apagados, $kT \ln 2$ cada. Conta-se sem
estimar:

2pt

- uma comparação lê dois valores de 64 bits e devolve um: apagam-se 127;
- uma escrita põe um valor onde estava outro: perdem-se os 64 que lá estavam;
- uma troca é involutiva --- aplicada duas vezes devolve --- e portanto não apaga nada .

~~@lrrrr@~~

~~$n=2 \times 10^5$ & bits a comparar & bits a escrever & trocas & bits/elem.~~

Dual Sort & 0 & 25 600 000 & 0 & 128,0

quicksort & 508 029 591 & 32 993 088 & 722 830 & 2705,1

mergesort & 415 599 245 & 452 845 568 & 0 & 4342,2

~~heapsort & 817 867 935 & 0 & 3 350 399 & 4089,3~~

Vinte e uma vezes menos que o quicksort, trinta e quatro que o mergesort --- e a coluna
que separa é a primeira: zero. Ele não faz uma única pergunta entre elementos.

E a latência, medida por fora

O sistema não ter tempo não dispensa medi-lo: a latência é da máquina, e mede-se por
fora . Ela não substitui a régua de cima a acompanha-a .

@lrrrr@

~~n=10^6, ms & Dual Sort & qsort & quicksort & mergesort & heapsort~~

uniforme & 16,0 & 98,3 & 57,6 & 73,8 & 107,7

quase ordenado & 7,6 & 34,3 & 18,8 & 27,0 & 54,9

invertido & 3,9 & 28,7 & 7,2 & 17,7 & 54,6

8 valores & 2,4 & 52,1 & 19,1 & 35,0 & 50,6

~~enviesado & 4,2 & 70,6 & 29,6 & 50,0 & 76,5~~

As duas tabelas não medem a mesma coisa : a de cima conta o que se apaga --- propriedade do algoritmo --- e esta conta quanto a máquina demora a fazê-lo. Um pode ser rápido e dissipar muito , e é o caso do quicksort: 7,4 ms no invertido, e 2705 bits por elemento.

E o invertido não é um caso onde o Dual Sort perca --- é onde a máquina custa mais. No corpo dual não há ordem privilegiada: crescente e decrescente são o mesmo objeto visto pelos dois lados, e o algoritmo faz exatamente o mesmo trabalho nos dois. Medido: 25 600 000 bits apagados numa lista invertida e 25 600 000 na mesma lista já crescente --- o número não muda. A diferença de milissegundos é decache , e é da máquina.

Ganha nos cinco. E o caso invertido, que numa versão anterior custava o dobro, passou a 3,9 ms --- porque essa versão descia níveis em vez de ler as marcas . Não foi uma otimização: foi deixar de perguntar. Não há motivo para uma realização ser diferente da outra , e não são.

Vale a pena dizê-lo porque é fácil de ler ao contrário : uma tabela de tempos sugere que há terreno favorável e desfavorável. Na régua do algoritmo não há --- ele apaga o mesmo em qualquer ordem de entrada.

O max-cut: ordenar e cortar são duais, e o corte é a massa

sec:cortar

Um caminho na árvore é um número escrito na base que ela define. Daí as duas leituras:

@l@

ordenar & dá a profundidade --- o caminho inteiro, todos os dígitos

~~cortar & dá a paridade --- um dígito, o último bit~~

Um lê o número, o outro escreve um bit dele: é o MOVE nos dois sentidos. Cada um sai do outro com resíduo 0, mas não são iguais --- são precisos n cortes para igualar uma ordem, e esse número é a profundidade.

E o corte constrói-se com meia volta

Toma-se a sequência natural e roda-se meia volta : [i (i + n/2) n] e o lado de cada elemento lê-se em quem atravessou : quem a rotação levou a dar a volta está de um lado, quem não deu está do outro. Essa sequência é o corte , e ele parte ao meio.

Meia volta é a velocidade máxima do relógio --- o único ponto onde ir e voltar são o mesmo caminho. Não há busca, não há comparação e não há escolha de centro: há uma rotação.

Massa \$=\$ corte

A segunda coordenada da cruz é o segundo momento . Postos os elementos em +1 e -1, com $|A|+|B|=n$: [corte = $|A| - |B|$, massa = $n - (|A|-|B|)^2/n$,] e as duas são função do mesmo desequilíbrio --- ambas máximas onde ele é zero. Não são duas quantidades que coincidem: são a mesma, e maximizar o corte é maximizar a massa.

E o que o corte carrega é o corpo ordenado

Não é um número que se extrai da sequência: o corte carrega o corpo ordenado na borda . A segunda coordenada da cruz é ordenada --- no bidual sempre existe ordem --- e uma ordem sobre n partes precisa de n componentes. Um escalar não a carrega: as 120 ordens de cinco partes dão uma só soma, e o número perde 119.

Por isso o corte não é o valor $|A||B|$: é a sequência de lados , uma componente por elemento, e $|A||B|$ é apenas um invariante dela. É a mesma razão por que a ordenação não produz a ordem --- ela já lá está, e o que se faz é lê-la.

Aqui esta quantidade chama-se corte ; no corpo estelar, onde a mesma álgebra mede o que fecha uma órbita, chama-se massa . É um objecto com dois nomes, um por domínio --- e nos dois é ordenado , porque é isso que o obriga a ter componentes.

O corte máximo numa árvore lê-se

A paridade da profundidade corta todas as arestas, porque toda a aresta liga um nó ao seu pai e pai e filho têm paridades opostas por construção. O máximo possível é o total, e a paridade atinge-o --a resposta lê-se, não se procura.

E a régua aqui é bits apagados, não passos. Este sistema é reversível e não tem tempo --- o que se mede como tempo é latência, e a latência não é propriedade do algoritmo. A pergunta ``quanto custa o corte?" responde-se em bits: o corte por rotação apaga zero, e a volta fecha com resíduo zero porque a mesma operação se desfaz a si própria. Procurar o melhor comparando candidatos apaga 63 bits por comparação. Não é que seja mais rápido: é que não paga. E o controlo diz de onde isso vem: num triângulo, o ciclo ímpar mais curto, nenhuma das oito bipartições corta as três arestas. É a árvore que o faz valer , e não esperteza do método.

numa árvore de 63 nós a paridade corta 62 de 62 arestas, o máximo possível; o comprimento do caminho iguala a profundidade em todos os nós; ordem $\rightarrow$ corte e caminho $\rightarrow$ ordem fecham com resíduo 0; são precisos 6 cortes para igualar a ordem e 6 é a profundidade; meia volta parte ao meio em cinco escalas sem excepção, sem uma comparação; corte e massa atingem o máximo no mesmo argumento em três escalas; e o controlo : no triângulo o melhor corte fica em 2 de 3; e o corte por rotação apaga 0 bits com a volta a fechar, contra 63 por comparação na busca (tests/maxcut.c).

A permutação é o dual, e a reversibilidade é o teste

O objeto não é a ordenação. Uma sequência é uma régua , e ordená-la é levá-la ao ponto fixo --- a forma canónica. Daí converte-se para qualquer outra: é isso a conversão entre régua, e é isso que se está a validar.

E o que valida não é a lista sair crescente: é a volta fechar. Ordenar sozinho deita fora de onde cada elemento veio. A permutação é o dual da ordenação --- ela guarda exatamente o que a projecção deitaria fora --- e com ela guardada o processo é invertível :

@lrr@

~~n=2×10⁵ & bits apagados & por elemento~~

sem guardar o caminho & 25 600 000 & 128,0

~~com o caminho guardado & 0 & 0,0~~

Zero. O que é reversível não apaga --- e a volta é exata em 200 000 de 200 000. O apagamento não era do algoritmo: era de não se guardar o dual.

E no banco não há array nenhum: há uma árvore

Pôr a sequência em disco não é guardá-la num ficheiro e mexer nela como se fosse memória. O banco é uma árvore , e ela está escrita: cada nó ocupa oito slots , com seis filhos por registo e o resto encadeado: [[n | resp] [s_1|f_1] ...[s_6|f_6] [0 | cont]] Uma fala desce por filho (no,símbolo), um símbolo por nível.

Aqui usam-se as mesmas peças, sem reescrever nenhuma:

@ll@

inserir & descer pelos símbolos do valor --- como a fala desce

ordenar & percorrer com os símbolos por ordem

~~repetidos & moram no contador do nó, e não descem outra vez~~

Sem baldes, sem tabelas, sem ficheiro temporário e sem uma única passagem sobre a sequência inteira . E os repetidos deixam de ser um caso a tratar: 500 cópias do mesmo valor ocupam 416 bytes, enquanto 500 valores distintos ocupam 74 912 --- 180 vezes mais . Não é uma optimização; é onde eles já estavam.

E a árvore não ordena sozinha: quem ordena é o percurso. Percorrida com os símbolos ao contrário, devolve a sequência decrescente, e nenhum par sobe. As duas saídas são os dois lados do mesmo percurso --- não há ordem privilegiada nem aqui.

entram 2000 e saem 2000, crescentes, com a soma conservada; 500 repetidos cabem em 416 bytes contra 74 912 de 500 distintos; o estado residente são 48 bytes e não vê o n; e o controlo : percorrida ao contrário não sobe um único ~~teste~~ (dualsort_banco.c).

Transformar uma sequência noutra

Nada de novo: é o mesmo A C=B do conversor, com a permutação no lugar do inverso. Levam-se as duas ao ponto fixo, e o conversor é a composição dos dois caminhos.

@l*12r@

A & 70 & 10 & 90 & 30 & 50 & 20 & 80 & 40 & 60 & 11 & 33 & 55

B & 5 & 4 & 9 & 1 & 7 & 2 & 8 & 3 & 6 & 12 & 10 & 11

~~C(A) & 33 & 30 & 60 & 10 & 50 & 11 & 55 & 20 & 40 & 90 & 70 & 80~~

C(A) tem a mesma ordem relativa que B --- os valores são os de A, a estrutura é a de B, e nenhum dos 132 pares discorda --- e são 132 e não 144: 12² conta a diagonal , e um elemento comparado consigo próprio concorda sempre. Doze casos que não podem falhar não medem nada , e inflavam a contagem em 9%. Ordenar é o caminho até à canónica; converter é ir de uma régua a outra por lá.

a volta é exata em 200 000 de 200 000, e com o caminho guardado apagam-se 0 bits contra 25 600 000 sem ele; e C(A) concorda com B em 132 de 132 pares --- sem a diagonal

(tools/bench_dissipa.c).

Por que pode ordenar sem saber nada

Porque a ordem não é produzida: já estava lá. Cada elemento traz a sua posição escrita nos símbolos, e o algoritmo lê o que ele já tem e põe-no onde já pertencia. Comparar é a pergunta de quem não lê --- e cada comparação apaga 127 bits para produzir um que estava disponível de graça.

O objetivo da ordenação não é criar ordem: é torná-la visível. É uma mudança de coordenadas --- da representação onde a ordem está no conteúdo para aquela onde está na posição . A informação é a mesma; muda onde se lê. Por isso os 384 bits são só escrita : não se pagou interrogação nenhuma, porque não houve nenhuma.

E o que se paga é irreduzível : depois de ordenar não há como saber de onde cada elemento veio, salvo guardando a permutação. Esse é o único apagamento que a ordenação exige ; o resto é o algoritmo a interferir por não ler.

$n=2 \times 10^5$, uniforme: 0 bits a comparar contra 508 milhões do quicksort; e a saída é verificada por três caminhos independentes --- crescente, igual elemento a elemento a uma ordenação por comparação, e a soma conservada (tools/bench_dissipa.c , tests/dualsort.c).

cinza Teoria, catálogo e medidores em
<https://goldenkingdom.patriatechnology.comgoldenkingdom.patriatechnology.com>. CC BY 4.0
(textos) · MIT (código).